Structured Light 3D Scanning By Means of a BYOP Machine Vision Camera

Daniel L. Lau, *Fellow, IEEE*

Abstract—Structured-light illumination (SLI) assumes that the camera integrates exactly one projected pattern per exposure. With an off-the-shelf digital light processing (DLP) projector that assumption is rarely checked: the projector re-times the video it receives, delays it by an unknown latency, and paints each frame as an interleaved sequence of colour-LED and bit-plane pulses whose timing is undocumented. We present a "bring your own projector" (BYOP) camera in which a field-programmable gate array sits inline on the HDMI cable between a PC and an unmodified projector. The same device reads the projector's EDID and drives it in a mode it advertises, generates resolution-adaptive phase-shift patterns on the fly, triggers a global-shutter PYTHON 1300 sensor at a programmable delay from the projected vertical sync with 10 ns resolution, and queues triggers so that patterns advance at the full video rate even when the delay exceeds a frame. Because the device controls both the picture and the shutter, it can also measure the projector: stepping a 4.875 μs exposure across a five-frame impulse sequence recovers the light the projector emits against time, which we use to place the exposure, to decompose white into its red, green and blue LED pulses, and to map the light that leaks into an exposure from the neighbouring patterns. On an Optoma ML750ST at 120 Hz we find a one-frame latency and a train of separate red, green and blue LED pulses whose filtered decomposition explains white with $R^2 = 0.9988$, and an exposure window that captures 95.6 % of a frame's light (99.2 % at the sensor's longest exposure); the reading before the window depends on light of the previous pattern in one part and, through the parasitic light sensitivity of the sensor's storage nodes, on the next pattern in another. [3D results: how much profiling and leakage correction improve phase and depth error.]

Index Terms—Structured light, phase measuring profilometry, projector–camera systems, DLP, genlock, FPGA, temporal characterization.

1 Introduction

Structured light illumination (SLI) refers to a method of 3D scanning that uses a projector to project a series of light striped patterns such that a camera can reconstruct depth based on the warping of the patterns over the target object's surface [1], [2], [3]. In this way, the projector takes the place of the second camera in a stereo pair, and the correspondence between a camera pixel and a projector row or column is established by the projected patterns rather than by the texture of the scene. Passive stereo vision, by comparison, must find correspondences from scene texture and fails on surfaces that have none [4], while time-of-flight (ToF) cameras measure depth directly from the round-trip time of modulated light but at a lower lateral resolution and with depth errors caused by multi-path reflections [5]. Laser line scanners, also based on triangulation, measure a single profile per captured image [6].

Examples of SLI include single pattern techniques which project a static pattern from which a 3D reconstruction can be made from a single snapshot [2], [7]. Multiple pattern SLI scanners, alternatively, project a series of patterns, trading temporal resolution for spatial resolution such that each pixel can be independently processed from its neighbors to produce a single point for each pixel in the camera. In phase measuring profilometry (PMP), the projector coordinate of each pixel is encoded through phase modulation over a minimum of three patterns, with multi-frequency schemes requiring more [3], [8]. As a scanning process, multiple pattern SLI assumes that the target object is stationary for the duration of the scan; when the object moves, the samples recorded at a pixel come from different surface points, and the reconstruction suffers from ripple and decoding errors [9], [10]. For this reason, it is desirable for the pattern sequence to run as fast as possible.

In practice, three factors limit the scan rate: the frame rate of the camera, the light output of the projector, and the bandwidth between the camera and the host computer. Each pattern must be captured in its own image, and machine vision cameras with megapixel global-shutter sensors operate at around 200 fps; the PYTHON 1300 sensor used in this paper operates at up to 210 fps at full resolution. DLP light engines built for structured light display binary patterns at tens of thousands of patterns per second, a rate that such cameras cannot use, and they are either expensive or have low lumen counts (Section 2.2). A projector with a low lumen count requires a longer exposure to collect the same signal, and since each exposure must end before the next pattern's exposure begins, the exposure time sets a lower bound on the time per pattern. Every image must also be transferred to the host: a 1280×1024 image at 10 bits per pixel is 1.64 MB, so 120 fps requires 197 MB/s of sustained bandwidth between the camera and the host.

For these reasons, commodity projectors are an attractive alternative. "3D-ready" DLP projectors accept 120 Hz video and can emit more than 1500 lumens, and if a light output below 1000 lumens is acceptable, then LED projectors can be used, whose LED lifetimes of tens of thousands of hours

• *D. L. Lau is with the Department of Electrical and Computer Engineering, University of Kentucky, Lexington, KY 40506, USA. E-mail: dllau@uky.edu*

make them practical for industrial applications (Section 2.2). Commodity projectors are, however, difficult to integrate into a 3D scanner. A commodity projector cannot store a static set of patterns in hardware, so every pattern must be sent over the video cable by a host computer whose graphics output makes no guarantee as to when each frame is delivered. A commodity projector also cannot be slaved to a camera acting as master through general purpose input/output (GPIO), nor can it trigger a camera, as structured light light engines do [11]. Of the published scanners built on commodity projectors (Section 2.3), only that of Yu *et al.* [12], [13] uses a controller fully interfaced between the HDMI video signal and the GPIO trigger of the camera.

The *bring your own projector* (BYOP) machine vision camera was developed to control an unmodified projector through its HDMI input [14], and it follows from the structured light smart camera for spatial augmented reality applications of Ruffner *et al.* [15]. The BYOP camera is a global-shutter machine vision camera, streaming full frames over USB 3.0, with an HDMI receiver and transmitter integrated on the same FPGA. Video from a host computer enters the camera and passes through to the projector, so the camera sits in the video path rather than beside it and knows what is being projected and when.

When a host computer drives the display, the camera either passes the host's pixels through untouched and, using the top-left pixel method of Yu [16], triggers only when the top-left pixel of the incoming frame changes, or replaces the host's pixels with structured light patterns of its own. When no host is attached, the camera reads the projector's EDID, drives the projector at a mode the projector advertises, generates its own pattern sequence, and triggers the sensor at a programmable delay after each vertical sync it transmits, at up to 120 Hz. No trigger box, frame grabber, or synchronization cabling is needed, and because the HDMI path runs through the camera, multiple BYOP cameras can be daisy-chained on a single projector.

Controlling when a frame is sent to the projector does not, by itself, control when or how the projector emits the light of that frame. Commodity projectors are designed to produce a compelling display for a human viewer watching movies, and the processing between the HDMI input and the emitted light serves that purpose rather than measurement. That processing delays each frame by an input lag that depends on the display mode, renders a constant input level as a temporal sequence of mirror states, applies a nonlinear tone curve, and can redistribute the RGB input among illumination components that the HDMI signal cannot address independently (Section 2.5).

An RGB LED projector also creates a problem of its own for SLI. A structured light pattern sent as gray levels, with equal red, green, and blue values, is projected as the sum of three patterns, one from each LED, and a pixel of a monochrome camera records each of them weighted by the reflectance of the surface in that LED's band, so that a green surface returns mostly the green pattern and a blue surface mostly the blue. If the three LEDs produce the same phase, the recorded phase is independent of the surface color. If they do not, because the LEDs have different tone curves or because the exposure collects a different share of each LED's light, then each pixel records a mixture of phases whose balance depends on its color, which is the same form as the multi-path interference studied by Zhang *et al.* [17], [18], and the error cannot be removed by a calibration made under the combined RGB light. Each LED must therefore be, by itself, a correct SLI projector that produces the same phase as the other two.

Projecting through only one LED does not avoid this requirement. Restricting the patterns to a subset of the red, green, and blue channels discards the light of the other LEDs and so reduces the light output of the projector, which is the property for which a commodity projector is chosen. A pattern sent as pure red, green, or blue also need not be displayed by one LED alone: the projector's color processing maps each input color to the drive of its light sources, in the manner of an ICC color profile, and can turn on more than one LED to display a single primary [19], [20] (Section 2.5).

This being the case, this paper studies the profiling of an LED commodity projector through the BYOP camera, with the aim of compensating the HDMI signal so as to minimize the distortion caused by the projector's electronics while achieving a 120 Hz scan rate.

1.1 Contributions

This paper is the first to describe the BYOP camera, and so the first contribution is the camera itself, implemented on a single Xilinx Artix-7 FPGA. On the video side, the FPGA includes an HDMI receiver that recovers the pixel clock of the incoming video by measuring its frequency and selecting the matching clock-recovery band, such that it tracks the spread-spectrum clocks of graphics cards, and an HDMI transmitter that either forwards the host's video or replaces it with patterns generated on the FPGA. The FPGA reads the projector's EDID over the display data channel and serves the host a merged EDID listing only the modes that both the projector and the camera support.

With no host attached, the FPGA reprograms its own pixel clock at run time and selects, from a table of 14 video modes, the fastest mode that the projector's EDID advertises, up to 1280×1024 at 60 Hz or 800×600 at 120 Hz. The on-board pattern generator produces PMP patterns at three spatial frequencies in a fixed 1:6:36 ratio with eight phase shifts each, plus a flat full-field pattern, with the periods recomputed for the resolution of each video mode, and passes every pattern through a 256-entry lookup table that can be loaded from the host.

On the camera side, the FPGA triggers the PYTHON 1300 sensor at a delay after the vertical sync of the video it transmits, programmable in 10 ns steps. In the controller of Yu *et al.* [13], the camera trigger waited out the trigger delay and the exposure before the next frame could be handled, so the fastest frame period was the trigger delay plus the exposure. The BYOP camera instead holds triggers in a 32-entry queue of absolute fire times, stacking a new trigger at every vertical sync while earlier ones are still pending, so that the frame period is limited only by the exposure and a delay longer than one frame period still yields one image per projected frame.

The FPGA limits the exposure to the frame period less the sensor's readout gap, receives the sensor's four LVDS lanes at 720 Mbps, buffers frames in DDR3 memory, and

streams them to the host over USB 3.0. In pass-through mode, the FPGA samples the top-left pixel of every incoming HDMI frame and triggers the camera when that value changes, and it writes the sampled value into the header of the captured frame, so that the host can match each captured image to the pattern that was displayed. The header also carries the mean intensity of a 16×16 pixel region of interest computed on the FPGA, which allows the projector profiling of this paper to run without transferring images.

While this paper limits its discussion to a single LED projector, the Optoma ML750ST, we note that the profiling procedure requires nothing from the projector other than an HDMI input, and so it can be applied to any projector that the BYOP camera can drive. Using the BYOP camera's own sensor, we sweep a 4.875 μs exposure across a five-frame impulse sequence to measure the light emitted by the projector against time, from which we derive the projector's latency, the timing of its LED pulses, and the exposure window that collects the most light from a single video frame.

By repeating the sweep through red, green, and blue filters, we test whether the projector's white output contains any interval during which two LEDs are on at once, which determines whether the three channels of the HDMI signal fully control the emitted light, and we measure the timing of each LED's light separately, which is the first requirement for the three LEDs to produce the same phase. [Per-LED phase test: scan a flat target through each filter and compare the three phase maps.]

We further measure the light that reaches the camera before the exposure window opens, and by means of two-dimensional maps over pairs of gray levels, separate the light arriving from the previous pattern from the light arriving from the pattern being captured. [Compensation of the HDMI signal from these measurements, and its effect on phase and depth error.]

2 Related Work

This section reviews the patterns of SLI and the projection hardware that displays them, then the ways that cameras have been synchronized with commodity projectors, the FPGA cameras that use the same sensor as the BYOP camera, the processing inside a projector that stands between its video input and its light, and finally multi-path, the class of errors to which that processing contributes.

2.1 Structured light and phase shifting

Pattern codification strategies for SLI are surveyed by Salvi *et al.* [1] and by Geng [2], and phase-shifting algorithms by Zuo *et al.* [3]. Liu *et al.* [8] combine a fine and a coarse stripe pattern in each image so that fewer patterns are needed to resolve which stripe a pixel sees, and Liu *et al.* [21] show that the projector's tone curve, which makes the displayed brightness grow faster than the input value, produces a repeating error in the measured phase. The pattern generator of the BYOP camera uses three stripe frequencies in a fixed 1:6:36 ratio, and it passes every pattern through a lookup table so that the projector's tone curve can be corrected before the patterns are sent.

High-speed SLI systems avoid the limits of video projection by displaying black-and-white patterns directly on a DMD. Zhang and Huang [22] as well as Zhang *et al.* [23] use this approach for real-time and superfast phase shifting, and Lei and Zhang [24] defocus the projector so that the black-and-white stripes blur into smooth gray-level stripes. Gong and Zhang [25] apply the same defocusing with an off-the-shelf DLP projector, and a review of these methods is given by Zhang [10]. These systems depend on projection hardware that exposes the DMD directly, which brings us to the light engines built for that purpose.

2.2 Light engines and commodity projectors

The DLP chipsets designed for high-speed pattern display operate far faster than any camera that watches them. The DLPC410 controller drives the DLP7000 digital micromirror device (DMD) at up to 23 000 black-and-white patterns per second over the full array, and at up to 48 000 patterns per second in a mode that repeats each row of the pattern four times and so gives up three quarters of the vertical resolution [26], [27]. Either rate exceeds the frame rate of a megapixel machine vision camera by more than two orders of magnitude.

Barreto describes the structured light modes, stored pattern sequences, and trigger outputs of TI's development kits [11]. These kits and the industrial light engines built on the same chipsets are either expensive or have low lumen counts. TI's DLP LightCrafter 4500, for example, displays up to 4225 black-and-white patterns per second from an RGB LED light engine rated at 150 lumens [28]. At the other extreme, TI's DLP Discovery 4100 platform, which displays binary patterns at up to 32 000 per second, consists of a controller board listed at about US$2400 and a DMD board listed at about US$2200 for the DLP7000 [29], [30], and these prices buy the mirrors and their electronics only: the platform includes no light source, illumination optics, or projection lens, which must be supplied separately [29].

Commodity projectors make the opposite trade, displaying patterns at a rate that machine vision cameras can match. "3D-ready" DLP projectors, built for shutter-glass stereo viewing, accept 120 Hz video and can have light outputs above 1500 lumens. As an example, ProjectorCentral measured 2494 lumens from the 3D-ready BenQ W600 in its brightest mode, and 1748 lumens in its standard mode [31]. The Optoma ML750ST used in this paper is an LED DLP projector with individual red, green, and blue LEDs and a rated LED life of 20 000 hours, and it accepts 800×600 video at 120 Hz [32], [33].

Optoma rates the ML750ST at 700 lumens, a figure that ProjectorCentral reports as the perceived equivalent of a lamp projector's lumens rather than a measured light output; ProjectorCentral measured 490 lumens in the ML750ST's brightest mode and 322 lumens in its cinema mode [34]. ProjectorCentral also lists the ML750ST's input lag, the delay between the arrival of a frame at the HDMI input and its appearance on screen, as 16.5 ms for 1080p video at 60 Hz [33].

We further note that faster scan rates are available from commodity 4K DLP projectors. TI's 0.47-inch 4K UHD DMD has 1920×1080 micromirrors and builds a 3840×2160 image

by shifting the picture among four positions with an optical actuator, and the chipsets built on this DMD accept 4K video at 60 Hz or 1080p video at 240 Hz [35], [36]. An SLI scanner built around a commodity 4K DLP projector could therefore scan at 240 Hz; such a scanner has not yet been demonstrated. Whatever its rate, a commodity projector accepts patterns only through its video input and offers no trigger, so a scanner built on one must synchronize its camera by other means.

2.3 Synchronizing commodity projectors and cameras

Examples of how to deal with this lack of synchronization include Wilm *et al.* [37], who insert fixed pauses between pattern presentation and image capture to accommodate buffering in the graphics card and projector, as well as Petković *et al.* [38], who use the blanking interval between video frames as a timing reference and estimate the combined projector–camera delay from captured images. Moreno *et al.* [39] take the approach of running projector and camera independently and computing, from the unsynchronized images, the images that a synchronized system would have captured. Zuo *et al.* [40], alternatively, modify a commodity projector with custom hardware that drives its DMD directly.

While each of these systems uses a commodity projector, none of them solves the problem of triggering the camera from the projector. To the best of our knowledge, Yu *et al.* [12], [13] are the first and only to demonstrate an SLI scanner with a controller fully interfaced between the HDMI video signal and the GPIO trigger of the camera.

Their controller operates either in an offline mode, in which it generates the video and patterns itself, or in a pass-through mode, in which it forwards video from a host computer; for their dual-projector scanner, both of the controller's HDMI ports were used as outputs, one to each projector [13]. As an example of a feature of the pass-through mode, Yu [16] describes reading the top-left pixel of each incoming frame and triggering the camera only when that pixel's value changes, so that frames repeated by the host's graphics processing unit (GPU) are not captured. Their controller waits out the trigger delay and the exposure before handling the next frame, a limitation analyzed in Section 3.4. That limitation follows from controlling an external camera through GPIO, and removing it means building the camera into the FPGA itself.

2.4 FPGA cameras built on the PYTHON 1300

Building the camera into the FPGA requires the FPGA to configure the image sensor itself, and the configuration of most image sensors is not public. The onsemi PYTHON 1300 is a 1.3-megapixel global-shutter CMOS sensor that reads out at up to 210 fps at its full 1280×1024 resolution over four LVDS data lanes, and it is configured over a serial peripheral interface (SPI). Its datasheet lists the sensor's registers, but not the values that make the sensor stream: the registers of the clock generator, image core, analog front end, bias, and LVDS outputs are listed only as "configure according to recommendation," the reserved registers must be loaded with values that are not given, and the datasheet states that the register uploads for each operating mode are available to customers under a non-disclosure agreement (NDA) [41]. An FPGA design that is to be published therefore cannot be written from the datasheet alone.

For this reason, we chose a sensor for which a working configuration had already been published with an FPGA design. Three such designs connect the PYTHON 1300 directly to an FPGA. Avnet publishes a reference design for its PYTHON-1300-C camera module, with files for several Zynq development boards, an SPI controller for the sensor, and the complete register upload sequence, including the sensor's timing program [42]. The Open Vision Computer of Quigley *et al.* [43] is an embedded vision system for mobile robots: its first revision carries three PYTHON 1300 sensors on an Artix-7 FPGA with DRAM and a USB 3 controller, and later revisions pair two PYTHON 1300 sensors with an Intel Cyclone FPGA and an NVIDIA Jetson TX2 [44].

Chalich *et al.* [45] built a low-cost, standalone high-speed camera around the monochrome PYTHON 1300 and a MicroZed board with a Zynq-7020, reaching 211 fps at full resolution and 2329 fps at 256×256, with frames stored in DDR3 memory and saved to an SD card, and they release the design as open source [46]. Each of these designs reads the sensor and stores or streams its images; none of them has a video interface or synchronizes the sensor to a projector. The BYOP camera's receive path was written for this work, using the Avnet register sequence and the pixel ordering of the Open Vision Computer's decoder as references.

2.5 Projector electronics

With the camera inside the video path, the remaining question is how the projector turns that video into light, since its electronics stand between the frame the camera knows about and the light the camera receives. Petković *et al.* [38] note that triggering a camera from the video signal ignores the projector's input lag, which they measured at 16.8 ms in one projector's 120 Hz 3D mode and 30–42 ms in its other modes. Narasimhan *et al.* [47] as well as Koppal *et al.* [48] show that a DLP projector displays a constant gray level as a rapid sequence of mirror flips between on and off, such that the light collected by a short exposure depends on when that exposure occurs within the frame.

The projector also changes what is displayed, not only when. Stone [19] describes the tone curves that projector electronics apply to the input, and she, as well as Seime and Hardeberg [49] and Wyble and Zhang [50], show that DLP projectors with a clear segment in their color wheel convert the input red, green, and blue values into red, green, blue, and white light.

Cotting *et al.* [51] measured the consequence directly, finding that for some pixel values a small change in one color changes the mirror sequence of the other colors; they measured these sequences first with a phototransistor and then with a camera whose exposure start time was swept across the frame. LED projectors eliminate the color wheel but not necessarily the problem, since DLP controllers can turn on two or more LEDs at once to produce yellow, cyan, magenta, or white intervals [20], light that cannot be controlled independently through the three color channels of an HDMI signal.

2.6 Light transport and multi-path

Light that reaches a camera pixel from more than one source is a familiar problem in SLI, although it is usually studied in space rather than in time. As an active imaging technique, structured light is susceptible to errors caused by the redirection of the projected light to form multiple paths from projector to target to camera besides the direct path, and the same problem is found in other 3D imaging modalities such as time-of-flight (ToF), where light reflects off specular surfaces onto neighboring surface points before returning to the camera.

Examples of how to deal with multi-path in ToF include Dorrington *et al.* [52] as well as Bhandari *et al.* [53] and Godbaz *et al.* [54], who take the common approach of measuring depth at many modulation frequencies and fitting the strength and delay of each path. Freedman *et al.* [55] assume that only a few paths contribute, Naik *et al.* [56] combine ToF measurements with a model of direct and indirect light, and Kadambi *et al.* [57], O'Toole *et al.* [58], and Gupta *et al.* [59] code the illumination in time to separate the paths.

In SLI, Nayar *et al.* [60] separate the light that travels directly from the projector to a surface point from the light that arrives indirectly, using finely striped illumination, Gupta *et al.* [61] design patterns that are robust to indirect light, and Gupta and Nayar [62] restrict phase shifting to a narrow band of fine stripe patterns. Chen *et al.* [63] modulate the phase-shifting patterns to reject light scattered beneath the surface, Couture *et al.* [64] replace striped patterns with random ones to overcome interreflections, and Dedrick [65] studies SLI in optically challenging environments. O'Toole *et al.* [66], [67], [68] synchronize the camera's exposure to the projector to select or reject particular light paths.

Zhang *et al.* [17], [18] address the case in which a camera pixel receives light from exactly two points on the projector, such as along a step edge, and separate the two by measuring how the two stripe signals reinforce and cancel as the stripe frequency changes. An RGB LED projector viewed by a monochrome camera produces a mixture of the same kind without any second surface. Each pixel records the sum of three stripe patterns, one per LED, each scaled by how strongly the surface reflects that LED's color. If the three LEDs place their stripes at the same position, the surface color changes only the contrast of the recorded stripes, and the measured phase is correct. If they do not, the measured phase is pulled toward the LED that the surface reflects most, so the same shape measures differently in different colors. Light from a neighboring pattern that falls inside the exposure, which this paper measures, adds a further contribution of the same kind, separated from the intended pattern in time rather than in space.

The three LEDs can also place their stripes at different positions because of the projection lens. Zhang, Towers, and Towers [69] show that lateral chromatic aberration gives the red, green, and blue channels of a color fringe projection system different phase maps, and they correct it from the difference between channels. Huang *et al.* [70] compare the shift between color channels seen on printed targets, which comes only from the camera's lens, against the shift seen on projected targets, which comes from both lenses, and find that the projector dominates in their DLP system, with a shift that changes with the distance to the target. Zhang *et al.* [71] separate the projector's shift from the camera's with an LCD phase target and attribute it to the projection lens: although every color is formed by the same DMD mirrors, the lens images each color at a slightly different position, by up to 1.96 pixels in their DLP projector.

How strongly each LED contributes is set by the target. Wang *et al.* [72] project red, green, and blue stripes with a DLP light engine onto colored surfaces viewed by a monochrome camera, observe that each colored region returns strong stripes only under the LED of its own color, and adapt the projected color and exposure to the surface; they attribute the wrinkles that remain in their reconstructions to the projector's tone curve. Zhu *et al.* [73] calibrate the projector's brightness response in advance to measure colorful objects.

The same dependence on surface color limits what a brightness calibration on a white target can establish. Projecting a series of gray levels onto a white target and recording them with a monochrome camera measures the sum of the three LEDs' responses, and a lookup table built from that measurement straightens only the sum. If the three LEDs have tone curves of different shapes, one LED's curvature can cancel another's on the white target and then return on a colored target that reflects one LED more than the others, distorting the recorded stripes. A single correction works for every target only when the three LEDs' tone curves have the same shape.

[Calibration [74]; event and motion-contrast scanners [75].]

3 The BYOP Camera

The BYOP camera is a stack of four functional circuit boards and two spacers, joined by Alchitry's board-to-board connectors, with no internal cables. Three of the functional boards are commercially available. The Alchitry Pt V2 carries a Xilinx Artix-7 XC7A100T FPGA, 256 MB of DDR3L memory, a 100 MHz oscillator, and 206 I/O pins [76]. Below it, the Alchitry Ft+ V2 provides a USB 3.0 link through an FTDI FT601Q bridge with a 32-bit parallel FIFO interface to the FPGA [77], [78], and below that, the Alchitry Hd V2 provides two HDMI ports, each configurable as an input or an output, which carry the video from the host computer and to the projector [79]. One spacer between the Pt V2 and the Ft+ V2 clears the capacitors on the underside of the Pt V2, and a second between the Ft+ V2 and the Hd V2 is needed because, without it, the Ft+ V2's USB connector blocks the cable of the Hd V2's transmitting HDMI port [76], [79].

The three boards must share the Pt V2's I/O pins without conflict, and each FPGA bank has a single I/O supply voltage. On the Pt V2, banks 14, 16, 34, and 35 are fixed at 3.3 V, bank 15 runs at 1.35 V for the on-board DDR3L, and bank 13 is the only bank whose voltage can be switched, between 3.3 V, 2.5 V, and 1.8 V [80], [81]. The Hd V2 uses pins A45–A78 of the bottom connectors, at 3.3 V, for its TMDS pairs and its DDC, CEC, and hot-plug lines, and the Ft+ V2 uses pins A3–A42 and B3–B24, also at 3.3 V, for the FT601's data bus and control lines; the two sets are disjoint, so these boards conflict only mechanically. The sensor's LVDS

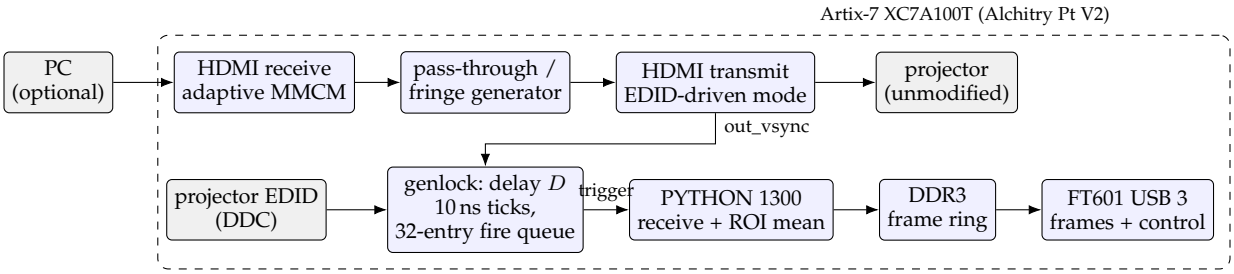


Fig. 1. The BYOP camera sits on the HDMI cable between the PC and the projector. Its FPGA either passes the PC's video through or replaces it with fringe patterns generated on the board, and it drives the projector in a video mode read from the projector's EDID. The camera's triggers are timed from the vertical sync of the *output* video. [Redraw: the EDID arrow should feed the transmit side; add the UART witness.]

lanes require 2.5 V, and the only pins that can run at that voltage are the 32 pins of bank 13, which appear only on the top connector, as pins B33–B78. Neither the Hd V2 nor the Ft+ V2 uses any of them, so the sensor card takes all 32, which is why it sits on top of the stack. For the same reason, the Alchitry Fn board cannot take the place of a spacer, because one of its solder jumpers lands on pin B36 and would take half of one LVDS pair.

An off-the-shelf alternative would combine an FPGA board that has HDMI ports with a camera module. Avnet's MicroZed Embedded Vision Kit, for example, pairs a Zynq-7020 module with a carrier card that has an HDMI output through an ADV7511 transmitter, an HDMI input through an ADV7611 receiver, and a connector for the same PYTHON-1300-C camera module whose register sequence we use [42], [82]. In its reference designs, the video timing is generated in the programmable logic, but the frames are read from memory buffers that Linux fills and selects, and the timing reaches the ADV7511 as sync codes embedded in the pixel data rather than as a vertical sync pin [42]; tying a frame's pattern to a camera trigger requires writing the video timing, pattern generation, and trigger logic in the FPGA fabric, which is the design this paper describes. The MicroZed's links to a host computer are also Gigabit Ethernet and USB 2.0, while 1280×1024 images at 10 bits per pixel and 120 fps require 197 MB/s. A camera module with a MIPI CSI-2 interface does not avoid these problems: the Zynq-7000 and Artix-7 have no hardened MIPI receiver, so the receiver must be written in the fabric with an external analog front end [83], and the global-shutter modules that a DLP projector requires have register settings that, like the PYTHON 1300's, are not public.

To interface the PYTHON 1300 to the FPGA, we designed a custom daughter card that mounts on top of the Pt V2, using the board schematics and the connector pin assignments that Alchitry publishes for designing boards that join its stack [80], [81]. The card carries the PYTHON 1300 (1280×1024, global shutter, four LVDS data lanes at 720 Mbps, 10-bit pixels) [41], routes its LVDS lanes to an FPGA bank powered at 2.5 V, and sits under a C-mount lens in a printed enclosure. The sensor is held in an Andon Electronics 48-pin LCC socket rather than soldered to the card. Because the monochrome and color versions of the PYTHON 1300 share the same 48-pin LCC package and pinout [41], the socket lets either version be inserted; the measurements in this paper use the monochrome NOIP1SN1300A. The footprint dimensions of the socket, which Andon's catalog does not give, were taken from the LUPA300 sensor carrier of Ruffner [84], [85], and we compared our decoupling against the same carrier.

The sensor board also generates the sensor's supplies, and the Pt V2 cannot supply them directly. The PYTHON 1300 requires 1.8 V, 3.3 V, and a pixel supply of 3.3 V held within 3.25–3.35 V, and it must be powered up in the order 1.8 V, 3.3 V, pixel supply, and down in the reverse order [41]. The Pt V2's 3.3 V rail is produced by an ADP5052 regulator whose feedback resistors set it to 3.278 V, with a tolerance of ±1.5 % [80], [86], so at its low limit it falls below the pixel supply's window. We therefore boost that rail to 4.45 V with a TPS61023 and regulate it back down with two TPS7A2033 low-dropout regulators, one for the 3.3 V supply and one for the pixel supply; a TPS7A2018 produces 1.8 V directly from the Pt V2's rail, and two TLV803S voltage supervisors enforce the power-up and power-down order [87], [88], [89].

Figure 1 shows how the FPGA design connects these boards. The HDMI datapath is adapted from [90], and the SLI pattern design follows the controller of Yu *et al.* [12], [13]. The following subsections describe the video path, the pattern generator, the genlock that times the camera to the projector, and the image and control path.

3.1 Video path

Algorithm 1 gives the procedure by which the BYOP camera detects the projector and learns which video modes it accepts. A projector lists those modes in its extended display identification data (EDID), a block of 128 or 256 bytes that a video source reads over the display data channel (DDC), an I^2C bus in the HDMI cable. The Hd V2 board does not connect the projector's hot-plug detect pin to the FPGA, so the BYOP camera detects the projector by whether its DDC answers. Every 0.5 s, the FPGA reads the projector's EDID; a read that is not acknowledged means that no projector is attached. After a successful read, the FPGA builds the EDID that it will present to a host computer, keeping only the projector's modes whose pixel clocks the receiver can recover, and writes it into the idle half of a double-buffered memory, so that a host never reads a partly built block.

Algorithm 2 then decides what the host sees and which source drives the projector. The FPGA raises the hot-plug signal on its HDMI input only when a projector is present and a merged EDID has been built, and whenever either

Algorithm 1 Projector detection and EDID merging

loop every 0.5 s
  read EDID block 0 from the projector's DDC at address `0xA0`
  if the read is not acknowledged **then**
    $present \leftarrow$ false ▷ no projector
    continue
  if byte 126 of block 0 is non-zero **then**
    read block 1 (CEA-861 extension)
  $present \leftarrow$ true
  $M \leftarrow$ projector's established and standard timings
  $M \leftarrow \{m \in M$: pixel clock of $m \in [25, 90]$ MHz$\}$
  if block 1 lists 1280×720@60 (VIC 4) **then**
    add it to M as a standard timing
  $E \leftarrow$ template EDID with timings M, a preferred detailed timing, range limits, and a name
  write a new serial number into E and recompute its checksum
  write E into the idle buffer; swap buffers
  $changed \leftarrow$ (E, less serial and checksum, differs from the last build)

condition changes, or the content of the merged EDID changes, it holds the signal low for 500 ms before raising it again. A host that sees the signal fall for longer than its debounce interval treats the display as unplugged and, when the signal returns, reads the new EDID; this matters for projectors that present a placeholder EDID while warming up and replace it with their real EDID seconds later. Each new build carries a new serial number, which prevents the host from reusing a cached copy.

These steps address flaws in the EDID handling of the controller of Yu *et al.* [12], [13], most of which follow from its hardware. Their HDMI shield wires the hot-plug signal of its input permanently to the computer's 5 V supply, so the FPGA cannot tell the computer to read the EDID again [91], and it connects the video lines to the FPGA without the 50 Ω termination to 3.3 V that an HDMI receiver presents; in our experience, computers with NVIDIA graphics cards did not detect it as a display. Its EDID was a fixed list of three modes chosen for one projector, and the projector's own EDID was never read. The Hd V2 terminates every video line and lets the FPGA drive the hot-plug signal through a level translator [92], which makes Algorithm 2 possible.

Algorithm 2 Host hot-plug and source selection

loop every clock cycle
  $sink \leftarrow present \wedge$ (a merged EDID has been built)
  if $sink$ changed $\vee$ $changed$ **then**
    $HPD \leftarrow 0$; $t \leftarrow 500$ ms
  else if $\neg sink$ **then**
    $HPD \leftarrow 0$
  else if $t > 0$ **then**
    $HPD \leftarrow 0$; decrement t
  else
    $HPD \leftarrow 1$ ▷ host reads the merged EDID
  if the input has a TMDS clock, its recovery MMCM is locked, and its symbols decode **then**
    drive the projector from the host's video ▷ pass-through
  else if decoding has been lost for 0.5 s **then**
    drive the projector from the FPGA's own timing ▷ offline

3.2 Pattern generator

In pass-through, the receiver deserializes and decodes TMDS and selects the recovery clock band from a measured input frequency, so the multiplier follows the source, and a high-bandwidth MMCM tracks spread-spectrum GPU clocks. With no PC, the device drives the projector itself: a dedicated MMCM is retuned at run time through its dynamic reconfiguration port and a timing generator is loaded from a table of 14 modes, with the projector's EDID acting as a filter on that table, never as a source of timings, and modes preferred by refresh rate and then by pixel count. The fastest modes in the table are 1280×1024@60 and 800×600@120 (CVT reduced blanking, 73.27 MHz), and all results in this paper use the second.

The two modes produce the same kind of pixel stream, a pixel clock with blanking, horizontal sync, vertical sync, and 8-bit red, green, and blue values, and the FPGA switches between them at a single point. In pass-through, the decoded pixels cross from the receiver's clock into the transmitter's clock through a small elastic buffer, since the two clocks have the same frequency but reach their circuits through different clock buffers, with a phase offset that the design does not control. Offline, a timing generator loaded with the selected mode produces the blanking and sync signals from the dedicated MMCM. A clock multiplexer chooses the receiver's clock only while the input has a clock, the receiver's MMCM is locked, and its symbols decode, and falls back to the offline clock once decoding has been lost for 0.5 s; a registered multiplexer selects the matching stream. Everything after that point, the pattern generator, the lookup table, the TMDS encoder and serializer, and the vertical sync from which the camera's trigger is timed, is the same circuit in both modes, so the patterns and the trigger timing do not depend on where the video came from.

In either mode, the pattern generator replaces the active pixels with PMP patterns whose parameters are derived from the resolution of the current video mode. The orientation of the stripes is set by the user from the host over USB, as are the color channels, red, green, or blue, that carry the pattern. Let F be the number of active pixels along the axis on which the patterns vary: the number of rows for horizontal stripes, or the number of columns for vertical stripes. The generator sets

$$b = \lceil F/288 \rceil, \qquad P_k = \frac{288\,b}{m_k}, \quad m_k \in \{1, 6, 36\}, \tag{1}$$

so that the three periods P_k are $288b$, $48b$, and $8b$ pixels, in the exact ratio 1:6:36, and the coarsest period spans the whole image. In our experience, about 36 periods across the field is the highest frequency that our scanners resolve reliably, and since each frequency's phase is unwrapped using the next lower one, we want the scale factor between successive frequencies to be as small as possible; with three frequencies reaching 36 periods, equal steps of 6 minimize

that factor. Every period is a multiple of 8, so one eighth of a period, $P_k/8 = 36b$, $6b$, or b pixels, is a whole number of pixels. For 800×600 with horizontal stripes, $F = 600$, $b = 3$, and the periods are 864, 144, and 24 rows.

The value of pattern $n \in \{0, \ldots, 7\}$ at frequency k and position y, the row or column index along the varying axis, is

$$I_{k,n}(y) = L\big[\, c\big((\phi_k(y) - 512\,n) \bmod 4096\big)\big]\,, \qquad (2)$$
$$c(a) = \Big\lfloor 255\,\big(\tfrac{1}{2} + \tfrac{1}{2}\cos\tfrac{2\pi a}{4096}\big) + \tfrac{1}{2}\Big\rfloor\,,$$

where c is a 4096-entry table of one cosine period, L is a programmable 256-entry lookup table that maps each 8-bit pattern level to the level sent to the projector, and $\phi_k(y)$ is the table address of position y, produced by a phase accumulator that advances by $m_k\lfloor 2^{24}/(288b)\rfloor$ per row or column and is read through its top 12 bits, so that it advances by 4096 entries over one period P_k. Because $\phi_k(y)$ does not depend on n, and 512 is exactly one eighth of the 4096-entry table, pattern n is pattern 0 shifted by exactly one eighth of a period: the eight patterns read the same table entries in the same order, and the rounding of the cosine and of the phase accumulator is identical in all eight, so it cancels when the phase is recovered.

The lookup table L is the point at which the BYOP camera compensates the HDMI signal for the projector's electronics. It is the identity at power-up, so that the projector receives the cosine levels unchanged, and the host can upload any 256-entry mapping over USB and read it back; Section 5 describes how the table is populated from measurements of the projector. One sequence is the three frequencies times eight phases, plus flat black and white frames for texture, which bypass L. For profiling, the same generator produces impulse sequences, cycles of five full-field frames with programmable levels (Section 4).

3.3 Image path

The FPGA configures the PYTHON 1300 over its serial peripheral interface (SPI), clocked at no more than 10 MHz, with a sequencer that walks the sensor's documented start-up flow. After pulsing the sensor's reset and confirming its chip identifier, the sequencer uploads the register sequence published by Avnet [42]: two stages of clock management with a wait for the sensor's PLL to lock, the required upload of reserved registers, the soft power-up that enables the LVDS drivers, and the sensor's 104-entry timing program, which sets how the pixel array samples and transfers each exposure.

The sensor runs in its PLL mode, in which the FPGA supplies a 72 MHz reference and the sensor multiplies it by five, so that it sends data on four LVDS lanes and a fifth sync lane at 720 Mbps, together with a forwarded 360 MHz clock. The FPGA receives the forwarded clock through an I/O clock buffer and a regional buffer that divides it by five, and deserializes each lane into 10-bit words at 72 MHz. While idle, the sensor sends the same training word on every lane, and the FPGA shifts the word boundary of each lane until that word appears, which aligns all five lanes independently.

The sync lane carries codes that mark the start and end of each frame and line and say whether the data lanes hold image pixels, black reference pixels, or a checksum. The data lanes deliver the pixels in kernels of eight, two words per kernel, with even kernels in ascending and odd kernels in descending order; the FPGA reorders each kernel and emits all eight pixels at once, so that it keeps pace with the sensor's four pixels per word clock, 288 million pixels per second. A full 1280×1024 frame therefore arrives in 4.55 ms, faster than USB can carry it, so each frame crosses into the clock of the DDR3 controller through a dual-clock FIFO and is written into a four-slot ring of frames in DDR3 memory.

A second dual-clock FIFO carries the frames from DDR3 to the FT601 bridge on the Ft+ V2, whose 32-bit FIFO interface runs on a 100 MHz clock that the bridge itself supplies. Each frame is sent as a header followed by its pixels, packed at 10 bits, four pixels in five bytes. The same USB link carries a command channel through which the host reads and writes the FPGA's registers and uploads the lookup table of Section 3.2. [Resource utilization of the current bitstream.]

The header carries everything that the host would otherwise have to infer about the frame, so that each frame describes itself. Its first 32 bytes begin with a marker word and end with the marker's bitwise complement, so that a host that loses its place in the stream can find the next frame, and between them hold the frame's rows and columns, its length in bytes, the pixel format, and a frame index that increments with every frame sent, so that a gap in the index reveals a lost frame. They also hold the number of frames that arrived short and were padded, the value of the top-left pixel of the HDMI frame being transmitted when the frame began, and the mean of a 16×16 pixel region of interest (ROI) at the center of the sensor, computed by the fabric from the frame's own pixels as they arrive, with the number of pixels averaged, a flag for an ROI that fell on the sensor's black reference rows, and a bit that marks whether a mean was paired with the frame at all. The profiling of Section 4 reads only this mean, so a sweep transfers no images.

When the host enables it, a second 32-byte block names the projected frame that the exposure captured: the number of projected frames since a reference frame, the frequency and phase of the pattern on screen in that projected frame, and flags for whether each value is known. The host sets the reference frame with a command that zeroes, on the same vertical sync, both this count and the pattern generator's sequence, so that frame zero always shows pattern zero. Section 3.4 describes how the label is carried from the trigger that opened the exposure to the frame's header. [The label block is implemented in the FPGA design and host software but not yet verified on hardware.]

3.4 Triggering with a long delay

The camera is synchronized to the projector by the vertical sync that the FPGA *sends*, so the timing reference is exact by construction, whether the video comes from a host or from the FPGA itself. A trigger fires a programmable delay D after that edge, counted in 10 ns ticks as a 24-bit value of up to 167.8 ms, and the exposure τ is set in units of 0.375 μs.

The controller of Yu *et al.* [13] triggers an external Allied Vision or Basler camera through GPIO, and the camera, not the controller, applies the trigger delay. The controller expects the camera to drive one of its output lines from

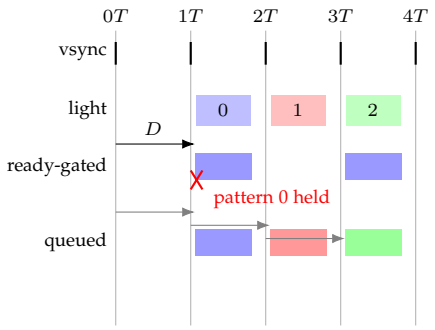


Fig. 2. Two ways to trigger the camera when the delay D is longer than the frame period T, drawn to scale for the ML750ST at 120 Hz ($D = 1.057\,T$, $\tau = 0.752\,T$). In ready-gated triggering, the camera is not ready at the vertical sync of frame 1, so the controller holds pattern 0 for a second frame and issues no trigger. In queued triggering, every vertical sync schedules a trigger at $kT + D$, and the pattern advances every frame.

its `FrameTriggerWait` signal, active while the camera waits for a trigger and inactive during its trigger delay and its exposure, and it reads that line as camera-ready. The controller issues a trigger only at a vertical sync that follows a rising edge of camera-ready, so the trigger of the next frame is withheld until the camera has finished its delay and its exposure.

Whether a scanner can run at the projector's full frame rate depends on when the light of each pattern reaches the camera relative to the vertical sync that sent it. A projector does not display a frame when it receives it: its electronics hold the frame for an input lag before the light leaves the lens, and the exposure that captures a pattern must wait for that light. To compare the two triggering schemes, let pattern k be sent with the vertical sync at time kT, and let its exposure open at $kT + D$ and close at $kT + D + \tau$. If D is shorter than T, each exposure opens before the next pattern is sent, and the camera is ready again in time. If D is longer than T, the exposure of pattern k opens after pattern $k + 1$ has already been sent, and the trigger for pattern $k + 1$ falls while the camera is still waiting out the delay of pattern k. The second case is the one that a commodity projector produces: for the ML750ST profiled in Section 4, the delay that collects the most light from a frame is about $1.06\,T$.

3.4.1 *Ready-gated triggering*

The controller of Yu *et al.* [13] connects to each external camera through four GPIO lines: a trigger output, a camera-ready input, a mode input that selects between pass-through and pattern generation, and an output that marks the first frame of the pattern sequence. The controller counts the rising edges of the camera-ready input. At each rising edge of the vertical sync, if a rising edge of camera-ready has arrived since the previous vertical sync, the controller advances the pattern index and pulses the trigger output during the vertical sync. If none has arrived, the controller holds the current pattern for another frame and issues no trigger.

The trigger therefore leaves the controller at the vertical sync, and the delay D is applied by the camera, which reports ready again only after its delay and exposure have ended. A pattern is consequently held on screen for

$$n_{\text{gated}} = \left\lceil \frac{D + \tau}{T} \right\rceil \tag{3}$$

frames, and the fastest frame period is the trigger delay plus the exposure. For the ML750ST, $(D + \tau)/T = 1.81$, so $n_{\text{gated}} = 2$: each pattern is displayed for two frames, and the scan rate falls from 120 Hz to 60 Hz. In pass-through mode the same handshake reaches the host, which waits for the camera to report ready before it plays the next frame.

The camera's side of the handshake is set up by the host software. For an Allied Vision camera, the software makes Line0 the trigger input, firing on a rising edge after a 10 μs debounce, and drives Line3 from `FrameTriggerWait`, so that the controller sees the camera become ready. Line1 is an output under software control that selects between pass-through and pattern generation; the software sets it low and then high to reset the controller's pattern counter and start the sequence, and low again when the scan ends. That reset is how the host knows which pattern each captured frame holds: after it, the nth frame captured holds the nth pattern of the sequence. Line2 is an input that reads the controller's first-frame marker. The camera captures a fixed number of frames equal to the length of the pattern sequence, with a trigger delay of 17.167 ms, one frame at 60 Hz plus 500 μs. When several cameras are used, all of them take the trigger, but only the first reports ready and controls the mode. A Basler camera is configured the same way on different lines, with Line1 as the trigger input and Line4 driven by `FrameTriggerWait`.

Because the controller counts rising edges of the ready line, a burst or a glitch on that line queues extra advances, and the pattern sequence then runs ahead of the captured frames by a fixed amount for the rest of the scan. Before integrating the sensor, we revised this logic so that at most one advance is owed at a time and a second ready edge before the next vertical sync is discarded, the first advance after the mode line rises requires a new low-to-high transition of the ready line, and the ready line must hold its level for about 135 μs before it is believed, since the camera cable is long and unshielded. These changes remove the failure, but not the limit of (3).

3.4.2 *Queued triggering*

The BYOP camera drives the PYTHON 1300 sensor directly, and the FPGA sets both the exposure and the delay, so there is no ready signal to wait for. The sensor operates in pipelined global shutter mode, integrating the next frame while it reads out the current one, and the pattern generator advances on every vertical sync. The FPGA keeps a free-running 25-bit counter of 10 ns ticks. At each vertical sync it writes the absolute fire time, the counter value at that sync plus D, into a 32-entry first-in first-out (FIFO) queue, whether or not earlier fire times are still waiting.

The FPGA fires the trigger at the head of the queue as soon as the counter reaches or passes its fire time, testing the difference of the two in modular 25-bit arithmetic, so a trigger cannot be skipped by missing an exact match. The number of triggers outstanding is counted with each push and pop resolved in the same clock cycle, and it is

reported to the host, which uses it to pair captured images with patterns: image n holds pattern $n - \lfloor D/T \rfloor$. A full queue increments an overflow counter rather than dropping a trigger silently.

With this queue, a trigger is never withheld, and the only requirement is that one exposure and the sensor's readout gap g fit within one frame period,

$$\tau + g \;\le\; T, \tag{4}$$

which the FPGA enforces by limiting the exposure to T less 90 μs, an 80 μs gap and a 10 μs margin. The delay itself is free: D may exceed several frame periods. Because the counter has one bit more than the delay, the modular comparison is valid for every delay the FPGA accepts, up to 2^{24} ticks (167.8 ms), at which point the queue holds $\lceil D/T \rceil = 21$ triggers at 120 Hz, within its 32 entries. For the ML750ST, $\lceil D/T \rceil = 2$ triggers are outstanding at a time, and the scan runs at the full 120 Hz.

Each of the four GPIO functions of the ready-gated design has an internal counterpart. The trigger is a signal inside the FPGA from the queue to the sensor, and the ready line is not needed, since the FPGA itself enforces (4); it remains as a register that the host can drive, so that the ready-gated pacing can still be tested without a camera. The selection between pass-through and pattern generation is a register written by the host over USB.

The reset line and the first-frame marker are replaced by the frame label of Section 3.3. The host's reset command zeroes the FPGA's count of projected frames and the pattern generator's sequence on the same vertical sync, and from then on every vertical sync has a number. Each entry of the trigger queue carries the number of the vertical sync that scheduled it, so the label leaves the queue with its own trigger and cannot drift from it. The FPGA records the pattern on screen in each projected frame in a 16-entry ring indexed by that number, and when the scheduled exposure ends, at the trigger time plus τ, it combines the two into the label. Because the next trigger cannot fire until one frame period later and τ is less than T by (4), the most recent trigger at that instant is always the one that opened this exposure. The label then travels with the frame's pixels into its header, so the host reads which pattern each frame holds from the frame itself rather than from the order in which frames arrive. [Measure: scan rate and image–pattern pairing with the queue versus ready-gated triggering on the ML750ST.]

4 Profiling the Projector

Before the BYOP camera can scan with a commodity projector, it must know when the light of each video frame leaves the projector, so that the trigger delay D and the exposure τ can be chosen to collect that light. To measure it, we built a profiling tool that uses the BYOP camera itself as the instrument. The tool drives the projector from the FPGA's pattern generator, sweeps a short exposure across the frame, and saves every sweep, together with the chosen delay and exposure, in a single document. All of the measurements in this section come from one such document, recorded with the Optoma ML750ST driven at 800×600 and 120 Hz with no host computer attached; the measured frame period is $T = 8325.81\,\mu s$.

For profiling, the sensor is mounted without a lens on the axis of the projection lens, about 17 mm in front of it in a printed cap, so that it receives light only when an LED is on and the corresponding mirrors are in their on state. An iris on the cap keeps the brightest reading below saturation, and filters can be placed in front of the sensor. Each reading is the mean of the 16×16 pixel region of interest that the FPGA computes and returns in the frame header, so a sweep transfers no images.

4.1 Sweeping the exposure

Let $E(t)$ be the light reaching the sensor at time t after the vertical sync of frame 0. An exposure of length τ opened at delay d reads

$$r(d) \;=\; g \int_{d}^{d+\tau} E(t)\,dt \;+\; o \;+\; n, \tag{5}$$

with gain g, an offset o, and noise n. A sweep of d in steps no longer than τ therefore samples E filtered by a box of width τ. The tool uses $\tau = 4.875\,\mu s$, a requested 5 μs exposure rounded to 13 of the sensor's 0.375 μs exposure units, and steps of 5 μs, for 1664 delays per frame. At each delay it discards four frames and averages the next 20.

The pattern generator repeats a cycle of five full-field frames, and the tool records the reading of every frame in the cycle, so a single sweep across one frame period yields a profile of all five frames. The first sequence we swept, *WWWWW*, shows white in every frame (255 in frame 0 and 254 in frames 1–4, so that the frame header still marks the start of the cycle). Its purpose was to see when the projector emits light. Figure 3 shows the result in blue: within each frame, the light arrives as a train of separate pulses, and the reading never returns to the camera's dark level of 18.33 ADU (its minimum is 30.25 ADU), so with a continuously lit picture there is no dark interval anywhere in the frame.

Because WWWWW repeats the same frame, its profile does not show where one frame's light ends and the next frame's begins. The second sequence, *WKKKK*, shows white in frame 0 and black in frames 1–4, so that only one frame of the cycle emits light. Its profile, in red in Fig. 3, contains the pulses that belong to the white frame, and they coincide with the WWWWW pulses. They appear not in frame 0 but in frame 1: the ML750ST displays each frame one frame period after it is sent. Frames 2–4 of WKKKK, far from any light, give the dark level $\bar{r}_0$ that is subtracted from every reading below.

Figure 4 enlarges Fig. 3 around the white frame's light, from the start of the runway to the trailing edge of the exposure (Sections 4.2 and 4.3), and adds, dashed, the WKKKK profile of frames 3 and 4 shifted back three frame periods. Because those frames are black and lie among black frames, the dashed line is the reading that an all-black KKKKK sequence would give. Two things can be seen by eye. First, inside frame 0, before the pulses of the white frame begin, the WKKKK reading ramps up from the dark level, reaching 75 ADU by the vertical sync of frame 1 and 87 ADU at the leading edge. We call this ramp the runway into the white

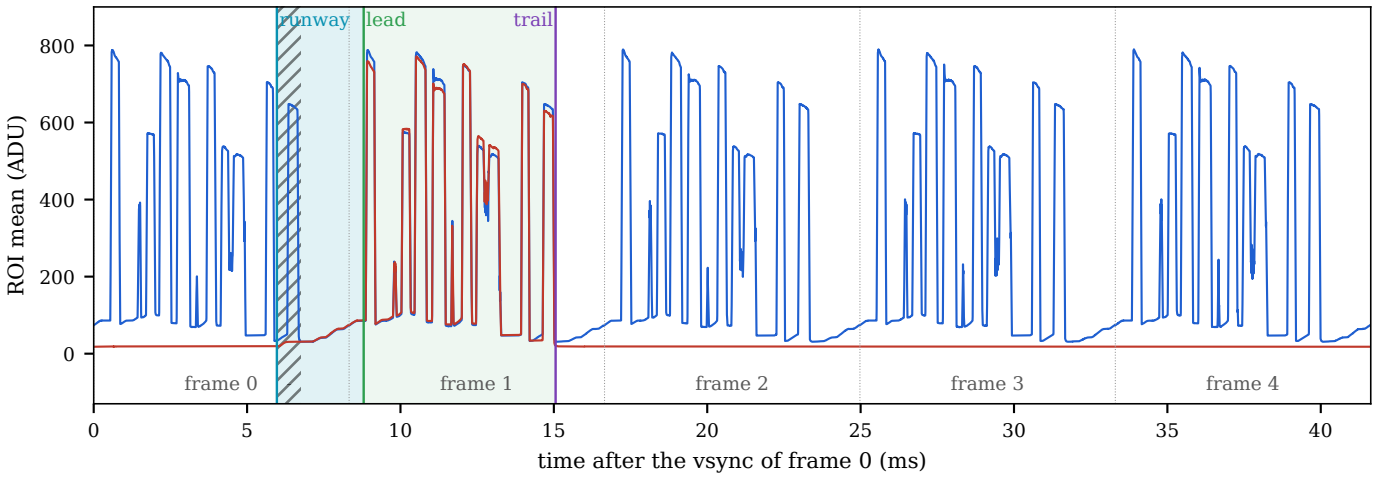


Fig. 3. Five-frame profiles of the ML750ST at 800×600 and 120 Hz, measured as the ROI mean with a 4.875 μs exposure swept in 5 μs steps. The blue WWWWW profile shows when the projector emits light, and the red WKKKK profile shows the light of the white frame 0, which appears in frame 1. The shaded regions mark the chosen exposure, from lead to trail, and the runway before it, and the hatched region marks the part of the runway that lies inside the previous exposure.

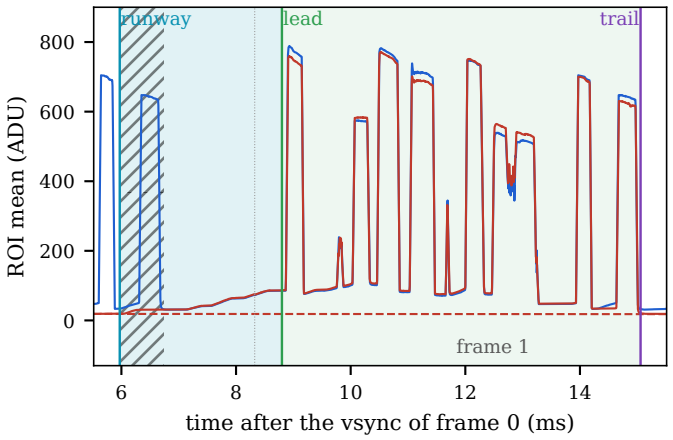


Fig. 4. A close-up of Fig. 3 from the start of the runway, ρ, to the trailing edge of the exposure, $s+\tau$, with WWWWW in blue and WKKKK in red. The red dashed line represents the output when all frames are black (K). The shaded regions mark the runway $[\rho,\ s]$ and the chosen exposure $[s,\ s+\tau]$, the hatched region marks the part of the runway inside the previous exposure, which ends at $s+\tau-T$, and the dotted line marks the vertical sync of frame 1.

frame. Section 4.3 attributes the ramp to the parasitic light sensitivity of the sensor, which adds light of the white frame that arrives after each exposure ends, and not to light that the projector emits during frame 0.

Without the white frame, in the KKKKK line, the same interval stays at the dark level, 18.2 ADU on average against 47.2 ADU for WKKKK. The ramp therefore depends on the frame that is displayed next, and it appears in readings whose exposures end before the first pulse of that frame. Second, the WWWWW profile shows that the runway also contains light of frame 0 itself: the last pulse of the frame displayed before the white one overlaps the start of the runway (hatched), and it is missing from WKKKK only because that frame is black there. The runway thus holds the late light of one frame and a reading that depends on the next frame, and for this reason we split it into two regions, $\mathcal{R}_1$ inside the previous exposure and $\mathcal{R}_2$ between exposures (Section 4.3). Figures 5 and 6 map, for each region, how the reading depends on the level of the runway's own frame and on the level of the frames around it (Section 4.4).

Figure 7 repeats Fig. 4 for the red, green, and blue profiles of the color sweeps (Section 4.5), and what Fig. 4 shows for white is true for all three colors, to varying degrees. Each profile ramps up through the part of the runway between exposures, from near the dark level at its start to 29.1, 53.6, and 41.0 ADU for red, green, and blue at the leading edge, so the ramp is strongest in green and weakest in red. The pulse of the prior frame that overlaps the start of the runway comes from one color alone: it is blue, and peaks at 654 ADU, while the red and green profiles stay within 13 ADU of the dark level there.

These observations rule out the usual approach to tone correction, in which the output of the projector is measured against the level of the current frame alone. Because the light collected during one frame's exposure also depends on the frame before it, the correction for the current frame must instead be measured as a function of the level of the previous frame, and because the effect differs from one LED to the next, it must be measured separately for red, green, and blue.

4.2 Choosing the delay and the exposure

With the white frame's light identified, the delay and exposure follow from it. Let $x(t) = \max(0,\ r_{\text{WKKKK}}(t) - \bar{r}_0)$. The share of one frame's light that an exposure $[s,\ s+\tau]$ collects is

$$C(s,\tau) \;=\; \frac{\int_s^{s+\tau} x(t)\,dt}{\int_0^{5T} x(t)\,dt}, \tag{6}$$

and the tool places the exposure at the start s that maximizes C for the longest exposure the sensor allows, after which the user can move either edge. The trigger delay is $D = s$. For the ML750ST, the edges saved in the document are $s = 8800.65\,\mu s$ and $s+\tau = 15057.48\,\mu s$, an exposure of $6256.83\,\mu$s that collects 95.6 % of the white frame's light.

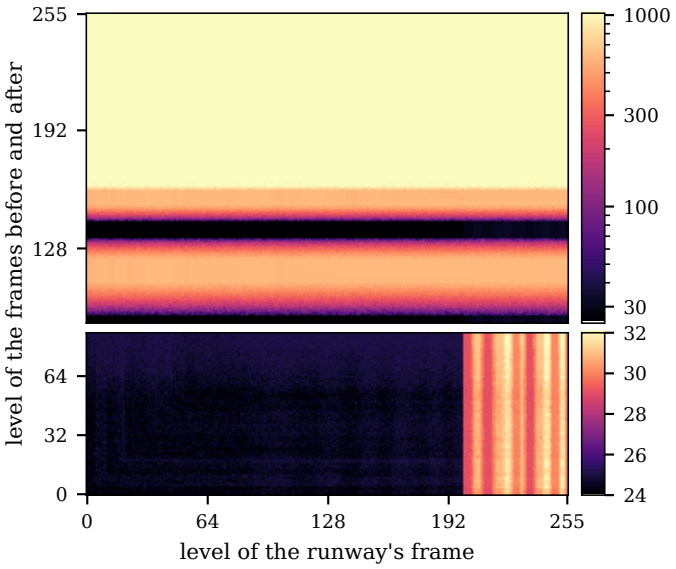


Fig. 5. The crosstalk map for $\mathcal{R}_1$, the part of the runway inside the previous exposure, 5971–6732 μs after the vertical sync of frame 0. Each cell is the mean ROI reading, in ADU, of 10 760.5 μs exposures of an A B A B cycle, and the map covers all 256×256 pairs of levels. The map is split at $y = 88$, with a log color scale above and a linear one below, where the reading stays between 24 and 32 ADU.

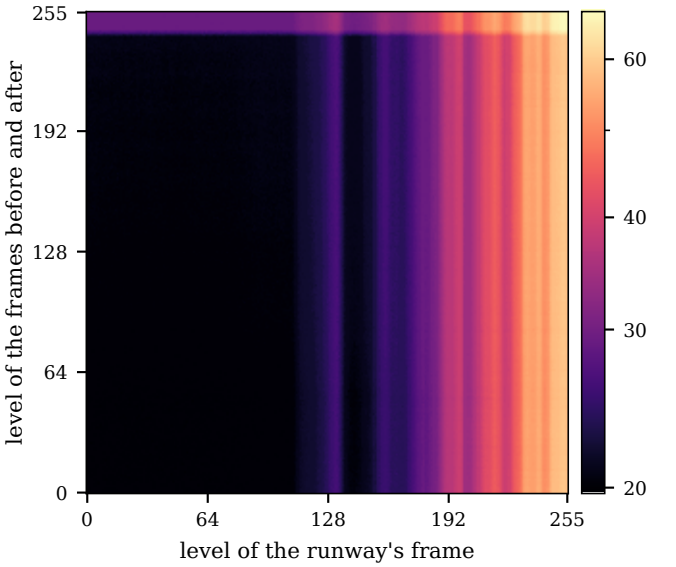


Fig. 6. The crosstalk map for $\mathcal{R}_2$, the part of the runway between exposures, 6732–8801 μs after the vertical sync of frame 0. Each cell is the mean ROI reading, in ADU, of 10 2068.9 μs exposures of an A B A B cycle, shown on a log color scale, and the map covers all 256×256 pairs of levels.

Since $D > T$, the exposure for each pattern opens after the vertical sync of the next pattern, which is the case analyzed in Section 3.4.

At the sensor's limit of 8235.75 μs, the best placed exposure collects 99.2 % of the frame's light. [Decide which exposure the scanning experiments use.]

4.3 The runway

The WKKKK sweep was intended only to locate the white frame's light, but it showed something we had not expected:

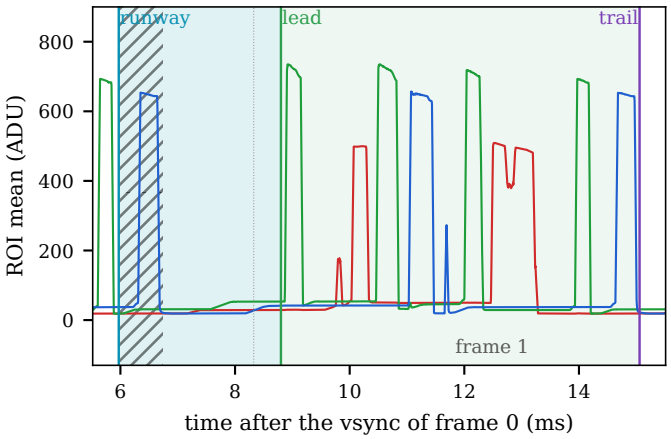


Fig. 7. The red, green, and blue profiles over the same interval as Fig. 4, from the start of the runway, ρ, to the trailing edge of the exposure, $s + \tau$. The shaded regions mark the runway $[\rho, s]$ and the chosen exposure $[s, s + \tau]$, the hatched region marks the part of the runway inside the previous exposure, and the dotted line marks the vertical sync of frame 1.

the difference between the white frame and the black frames extends beyond one frame period. The WKKKK reading rises above the dark level before the leading edge of the exposure window, in an interval we call the *runway*, $[\rho, s]$ with $\rho \geq s - T$. The tool places ρ by walking back from s while $x(t)$ remains above 2 % of its peak, which for the ML750ST gives $\rho = 5970.99\,\mu s$ (Fig. 4).

The PYTHON 1300 is a global shutter sensor with a storage node inside each pixel. At the end of an exposure, the sensor transfers the charge of every pixel to its storage node at the same time, and it then reads the pixel array out one row at a time [41]. The charge of a row therefore waits on the storage node until that row is read. A storage node is shielded from light but not completely: it collects a small fraction of the light that reaches the pixel, through light that enters at an angle under the shield and through charge that diffuses from the photodiode [93], [94]. The ratio of the sensitivity of the storage node to that of the photodiode is the parasitic light sensitivity (PLS) of the pixel, and its inverse is the global shutter efficiency [93], [94]. The PYTHON 1300 datasheet gives a PLS below $1/8000$ [41], and PLS varies with the wavelength and the angle of incidence of the light [95].

With a storage node that waits a time W before its row is read, the model of (5) becomes

$$r(d) \;=\; g \int_{d}^{d+\tau} E(t)\,dt + g\,p \int_{d+\tau}^{d+\tau+W} E(t)\,dt + o + n, \quad (7)$$

where p is the PLS. The second integral weights light that arrives after the exposure ends, and its share of the reading grows with W/τ. For the sweeps, $\tau = 4.875\,\mu s$, and with W in the milliseconds the second integral spans hundreds of exposure lengths. In the WKKKK sweep, an exposure inside the runway collects no light of its own, and the second integral covers the first pulses of the white frame. The ramp begins at ρ, 2920 μs before the first pulse of the white frame reaches half of its peak at 8890.81 μs, which is consistent with $W \approx 2.9\,ms$. [Measure W and p with an exposure-

length test in the profiling tool, and confirm with a power meter that the projector emits no light during the runway.]

For the exposure chosen in Section 4.2, $\tau = 6256.83\,\mu s$, the same W spans less than half of one exposure length. For light of equal brightness inside and after the exposure, the second integral of (7) is then pW/τ of the first, which is below 6×10^{-5} for $p < 1/8000$. The runway is therefore large only for the short exposures used for profiling. [Replace the datasheet bound with the measured p.]

The runway overlaps the exposure of the previous pattern, which ended at $s + \tau - T = 6731.67\,\mu s$, and within that overlap lies a pulse of light from the prior frame. The runway therefore divides into two parts,

$$\mathcal{R}_1 = [\rho,\ s + \tau - T], \qquad \mathcal{R}_2 = [s + \tau - T,\ s], \tag{8}$$

the part inside the previous exposure, 760.7 μs long, and the part between exposures, 2069.0 μs long. They hold 0.35 % and 3.65 % of the integral of $x(t)$ over the cycle, respectively.

4.4 Crosstalk maps

To find whose light arrives in each part of the runway, the pattern generator switches to a two-frame cycle $A\,B\,A\,B$ of full-field levels. A reading in frame k then sees that frame's level x and the level y of frames $k \pm 1$, which are equal. For each part $\mathcal{R}_i$, the tool opens an exposure at the start of the part, lasting its length, and records 10 readings for every pair (A, B); each pair fills the two cells $(y, x) = (B, A)$ and (A, B) of a 256×256 map. A map that varies with y shows light from the neighboring patterns, and one that varies with x shows light from the runway's own frame. Because $A\,B\,A\,B$ makes the previous and the next frame equal, the maps cannot tell those two apart.

The two maps, Figs. 5 and 6, behave differently. In $\mathcal{R}_1$, the reading is set by the level y of the neighboring frames and spans 24 to more than 1023 ADU as y changes, but not monotonically: it stays near 24 ADU up to $y \approx 88$, rises in a band near $y = 96$–120, returns to the floor near $y = 136$, and saturates from about $y = 164$. In $\mathcal{R}_2$, the reading is set instead by the runway frame's own level x, rising from 20.0 to 57.2 ADU across x at $y = 0$, with a step of about 8 ADU between $y = 240$ and $y = 248$. Read with the one-frame latency, $\mathcal{R}_1$ contains light of the previous pattern, which the previous exposure is meant to collect and the current one must not. The reading in $\mathcal{R}_2$ depends instead on the pattern about to be captured. The exposure of $\mathcal{R}_2$ ends at s, and by (7) its reading includes the light of that pattern's first pulses, which arrive within W after s. [Interpret the non-monotonic dependence on y; test whether the order of the prior frame's bit-planes explains it.]

Over most of their range, each map therefore depends on one level only. The map of $\mathcal{R}_2$ is constant along y for $y < 248$, and the map of $\mathcal{R}_1$ is constant along x for $x < 193$. Above that level, the reading in $\mathcal{R}_1$ rises by 5 to 8 ADU at every value of y that does not saturate the sensor, from 24.7 to 29.0 ADU at $y \le 88$. This step is the one place in either map where the reading depends on the level of a second frame, and it is small against the light of the previous pattern in the same interval. On the edges of the bands in y, near $y = 96$, 128, and 144, the reading in $\mathcal{R}_1$ also varies by up to about 100 ADU across x.

4.5 Color sweeps

The pulse that falls inside $\mathcal{R}_1$ raises the question of which LED emits it, and so we repeated the WWWWW sweep through red, green, and blue filters on the camera. With $w(t)$ and $f_c(t)$ the white and filtered sweeps less $\bar{r}_0$, the tool fits

$$w(t) \approx \sum_{c\in\{R,G,B\}} a_c\, f_c(t) + b \tag{9}$$

by least squares over every delay, where the weights a_c absorb the transmission of each filter. For the ML750ST the fit gives $a_R = 0.4935$, $a_G = 1.2192$, $a_B = 0.6945$ and $b = -3.28$ ADU, with a residual of 9.61 ADU RMS against a 771.7 ADU peak and $R^2 = 0.99880$ (Fig. 8). The white output of the projector is therefore the sum of its red, green, and blue pulses, with no interval in which a fourth component, such as white from two LEDs at once, contributes light that the three filtered sweeps do not account for. The largest residuals sit on pulse edges, where a small timing difference between sweeps leaves a spike, and inside the long red pulse. [The red sweep may be partly clipped; repeat it with the iris closed one step.]

The color sweeps also show that the red, green, and blue pulses occupy separate intervals of the frame, and that the pulse inside $\mathcal{R}_1$, near the end of the frame, is blue (Fig. 7). A scan made through a red or green filter on the camera lens would therefore receive no light from the prior frame inside its exposure, so that a target can be scanned without this crosstalk by choosing the filter on the camera. The filter must be on the camera and not the projector's video: sending a single primary over HDMI does not guarantee that only one LED emits light, since the projector's color processing can drive other LEDs from a pure primary input (Section 2.5), whereas a filter on the camera lens passes only the light of one LED whatever the projector does. The cost of the filter is the light of the other two LEDs, which lengthens the exposure.

These measurements give the delay and exposure for the scanner, and they identify where its crosstalk comes from. Whether the scanner then measures phase correctly is the subject of the next section.

5 Experiments

[Scanning experiments, no results yet. They measure how well the scanner recovers phase with the delay and exposure chosen in Section 4. (1) Phase error on a flat target with the 24-frame PMP sequence at 120 Hz, for an exposure placed at delay 0, at the profiled window, and at the sensor's exposure limit. (2) The same scans through red, green, and blue filters on the camera lens, to test whether each LED alone produces the same phase, at the center and at the edge of the field, and whether a filter removes the crosstalk from the prior frame. (3) The response through each filter on red, green, blue, and white targets after a lookup table linearizes the gray-level response on a white target. (4) Phase error with pattern levels above 192, where the next frame adds 5 to 8 ADU inside $\mathcal{R}_1$, against patterns held below 193. (5) Scan rate and image–pattern pairing with queued versus ready-gated triggering. (6) Depth error on calibrated planes and a sphere. (7) Populating the lookup table L of (2) from measurements of the projector, and its effect on phase error.]

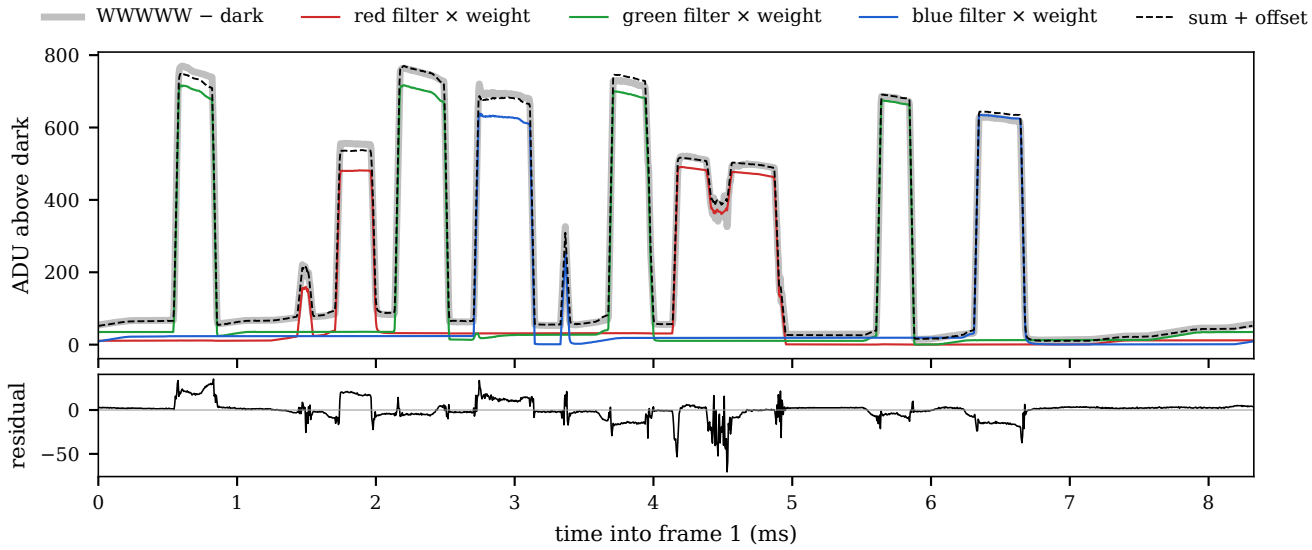


Fig. 8. The white sweep, less the dark level, over one frame (gray band), fitted as a weighted sum of the red-, green-, and blue-filtered sweeps by (9). The lower panel shows the residual of the fit.

6 Limitations

The profiles are ROI means over a 16×16 patch with the sensor at the lens, not images of a scene; a pixel in a scanned scene integrates the same pulse train through a different optical path. The A B A B cycle confounds the previous and the next pattern. One projector has been measured.

7 Conclusion

Putting the camera inside the HDMI path turns an unmodified projector into an instrument whose timing can be measured rather than assumed. The same genlocked exposure that captures structured light, stepped through an impulse sequence, recovers the projector's latency, its LED pulse train, an exposure window that collects nearly all of a frame's light, the light of the previous pattern that falls before that window, and the reading that the parasitic light sensitivity of the sensor adds from the next pattern. [Summarize the 3D results.]

Acknowledgments

The HDMI datapath derives from M. Field's Artix-7 HDMI processing design [90]; the SLI controller design follows Yu *et al.* [13]. [Funding.]

References

[1] J. Salvi, J. Pagès, and J. Batlle, "Pattern codification strategies in structured light systems," *Pattern Recognition*, vol. 37, no. 4, pp. 827–849, 2004.

[2] J. Geng, "Structured-light 3D surface imaging: a tutorial," *Advances in Optics and Photonics*, vol. 3, no. 2, pp. 128–160, 2011.

[3] C. Zuo, S. Feng, L. Huang, T. Tao, W. Yin, and Q. Chen, "Phase shifting algorithms for fringe projection profilometry: A review," *Optics and Lasers in Engineering*, vol. 109, pp. 23–59, 2018.

[4] D. Scharstein and R. Szeliski, "A taxonomy and evaluation of dense two-frame stereo correspondence algorithms," *International Journal of Computer Vision*, vol. 47, no. 1–3, pp. 7–42, 2002.

[5] M. Hansard, S. Lee, O. Choi, and R. Horaud, *Time-of-Flight Cameras: Principles, Methods and Applications*, ser. SpringerBriefs in Computer Science. Springer, 2012.

[6] F. Blais, "Review of 20 years of range sensor development," *Journal of Electronic Imaging*, vol. 13, no. 1, pp. 231–240, 2004.

[7] M. Takeda and K. Mutoh, "Fourier transform profilometry for the automatic measurement of 3-D object shapes," *Applied Optics*, vol. 22, no. 24, pp. 3977–3982, 1983.

[8] K. Liu, Y. Wang, D. L. Lau, Q. Hao, and L. G. Hassebrook, "Dual-frequency pattern scheme for high-speed 3-D shape measurement," *Optics Express*, vol. 18, no. 5, pp. 5229–5244, 2010.

[9] T. Weise, B. Leibe, and L. Van Gool, "Fast 3D scanning with automatic motion compensation," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2007.

[10] S. Zhang, "High-speed 3D shape measurement with structured light methods: A review," *Optics and Lasers in Engineering*, vol. 106, pp. 119–131, 2018.

[11] R. Barreto, "Using DLP® development kits for 3D optical metrology systems," Texas Instruments, Application Report DLPA026, 2011.

[12] Y. Yu, D. L. Lau, and M. Ruffner, "3D scanning by means of dual-projector structured light illumination," in *Emerging Digital Micromirror Device Based Systems and Applications XI*, ser. Proc. SPIE, vol. 10932, 2019, p. 109320M.

[13] Y. Yu, D. L. Lau, M. P. Ruffner, and K. Liu, "Dual-projector structured light 3D shape measurement," *Applied Optics*, vol. 59, no. 4, pp. 964–974, 2020.

[14] D. L. Lau, "University of Kentucky researchers develop "bring your own projector" machine vision camera," Press release, University of Kentucky, 2026, [cite the published Quality Magazine article instead?]

[15] M. Ruffner, Y. Yu, and D. L. Lau, "Structured light smart camera for spatial augmented reality applications," in *Emerging Digital Micromirror Device Based Systems and Applications XI*, ser. Proc. SPIE, vol. 10932, 2019.

[16] Y. Yu, "Spatial augmented reality using structured light illumination," Ph.D. dissertation, University of Kentucky, 2019, https://uknowledge.uky.edu/ece_etds/143.

[17] Y. Zhang, D. L. Lau, and Y. Yu, "Causes and corrections for bimodal multi-path scanning with structured light," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 4431–4439.

[18] Y. Zhang and D. L. Lau, "BimodalPS: Causes and corrections for bimodal multi-path in phase-shifting structured light scanners," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 6, pp. 4001–4017, 2024.

[19] M. C. Stone, "Color balancing experimental projection displays," in *Proc. Ninth IS&T/SID Color Imaging Conference*, 2001.

[20] Texas Instruments, "DLP® pico LED driver design guide and LM3434," Texas Instruments, Application Report DLPA038A.

[21] K. Liu, Y. Wang, D. L. Lau, Q. Hao, and L. G. Hassebrook, "Gamma model and its analysis for phase measuring profilometry," *Journal of the Optical Society of America A*, vol. 27, no. 3, pp. 553–562, 2010.

[22] S. Zhang and P. S. Huang, "High-resolution, real-time three-dimensional shape measurement," *Optical Engineering*, vol. 45, no. 12, p. 123601, 2006.

[23] S. Zhang, D. Van Der Weide, and J. Oliver, "Superfast phase-shifting method for 3-D shape measurement," *Optics Express*, vol. 18, no. 9, pp. 9684–9689, 2010.

[24] S. Lei and S. Zhang, "Flexible 3-D shape measurement using projector defocusing," *Optics Letters*, vol. 34, no. 20, pp. 3080–3082, 2009.

[25] Y. Gong and S. Zhang, "Ultrafast 3-D shape measurement with an off-the-shelf DLP projector," *Optics Express*, vol. 18, no. 19, pp. 19 743–19 754, 2010.

[26] *DLPC410 DMD Digital Controller*, Texas Instruments, 2020, datasheet DLPS024G, Aug. 2012, revised Feb. 2020; Table 19.

[27] *DLP7000 DLP® 0.7 XGA 2x LVDS Type A DMD*, Texas Instruments, 2023, datasheet DLPS026G, Aug. 2012, revised Apr. 2023.

[28] Texas Instruments, "DLPLCR4500EVM: DLP LightCrafter 4500 evaluation module," https://www.ti.com/tool/DLPLCR4500EVM, accessed 2026-10-23.

[29] ——, *DLP Discovery 4100 Development Platform: EVM User's Guide*, Texas Instruments, dLPU040D, revised August 2026.

[30] Mouser Electronics, "DLPLCRC410EVM and DLPLCR70EVM evaluation module listings," https://www.mouser.com, accessed 2026-10-23; single-unit prices of US$2399.16 and US$2158.89 on regional storefronts.

[31] B. Livolsi, "BenQ W600 720p 3D DLP projector review," ProjectorCentral, https://www.projectorcentral.com/benq_w600_projector_review.htm, Mar. 2010.

[32] Optoma, "ML750ST palm-sized LED short throw projector," https://www.optomausa.com/product/ml750st, product page, accessed 2026-10-20.

[33] ProjectorCentral, "Optoma ML750ST projector specifications," https://www.projectorcentral.com/optoma-ml750st.htm, accessed 2026-10-20.

[34] M. D. Stone, "Optoma ML750ST projector review," ProjectorCentral, https://www.projectorcentral.com/optoma-ml750st-projector-review-road-test.htm, Jul. 2016.

[35] *DLP471TE 0.47-Inch 4K UHD Digital Micromirror Device*, Texas Instruments, 2025, datasheet DLPS170C, Sept. 2020, revised Jan. 2025.

[36] *DLPC6540 Display Controller*, Texas Instruments, 2022, datasheet DLPS168C, May 2021, revised Nov. 2022.

[37] J. Wilm, O. V. Olesen, and R. Larsen, "Accurate and simple calibration of DLP projector systems," in *Proc. SPIE*, vol. 8979, 2014, p. 897909.

[38] T. Petković, T. Pribanić, M. Đonlić, and N. D'Apuzzo, "Software synchronization of projector and camera for structured light 3D body scanning," in *Proc. 7th International Conference on 3D Body Scanning Technologies*, 2016, pp. 286–295.

[39] D. Moreno, F. Calakli, and G. Taubin, "Unsynchronized structured light," *ACM Transactions on Graphics*, vol. 34, no. 6, pp. 178:1–178:11, 2015.

[40] C. Zuo, Q. Chen, G. Gu, S. Feng, and F. Feng, "High-speed three-dimensional profilometry for multiple objects with complex shapes," *Optics Express*, vol. 20, no. 17, pp. 19 493–19 510, 2012.

[41] *PYTHON 1.3/0.5/0.3 MegaPixels Global Shutter CMOS Image Sensors (NOIP1SN1300A)*, onsemi, 2021, datasheet NOIP1SN1300A/D, Rev. 6, Aug. 2021, https://www.onsemi.com/download/data-sheet/pdf/noip1sn1300a-d.pdf.

[42] Avnet, "Avnet HDL repository: PYTHON-1300-C camera module reference design (`fmchc_python1300c`) and `onsemi_vita_spi` core," https://github.com/Avnet/hdl.

[43] M. Quigley, K. Mohta, S. S. Shivakumar, M. Watterson, Y. Mulgaonkar, M. Arguedas, K. Sun, S. Liu, B. Pfrommer, V. Kumar, and C. J. Taylor, "The Open Vision Computer: An integrated sensing and compute system for mobile robots," in *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, 2019, pp. 1834–1840.

[44] Open Source Robotics Foundation, "Open Vision Computer (OVC) hardware, firmware, and software," https://github.com/osrf/ovc, apache-2.0.

[45] Y. Chalich, A. Mallick, B. Gupta, and M. J. Deen, "Development of a low-cost, user-customizable, high-speed camera," *PLoS ONE*, vol. 15, no. 5, p. e0232788, 2020.

[46] Y. Chalich, "HFRC: High frame rate camera project for a MicroZed 7020 SoC with the PYTHON 1300," https://github.com/yamnchalich/HFRC, 2020, gPL-3.0.

[47] S. G. Narasimhan, S. J. Koppal, and S. Yamazaki, "Temporal dithering of illumination for fast active vision," in *Proc. European Conference on Computer Vision (ECCV)*, 2008, pp. 830–844.

[48] S. J. Koppal, S. Yamazaki, and S. G. Narasimhan, "Exploiting DLP illumination dithering for reconstruction and photography of high-speed scenes," *International Journal of Computer Vision*, vol. 96, no. 1, pp. 125–144, 2012.

[49] L. Seime and J. Y. Hardeberg, "Colorimetric characterization of LCD and DLP projection displays," *Journal of the Society for Information Display*, vol. 11, no. 2, pp. 349–358, 2003.

[50] D. R. Wyble and H. Zhang, "A colorimetric characterization model for DLP projectors," in *Proc. Eleventh IS&T/SID Color Imaging Conference*, 2003, pp. 346–350.

[51] D. Cotting, M. Naef, M. Gross, and H. Fuchs, "Embedding imperceptible patterns into projected images for simultaneous acquisition and display," in *Proc. Third IEEE and ACM International Symposium on Mixed and Augmented Reality (ISMAR)*, 2004, pp. 100–109.

[52] A. A. Dorrington, J. P. Godbaz, M. J. Cree, A. D. Payne, and L. V. Streeter, "Separating true range measurements from multi-path and scattering interference in commercial range cameras," in *Three-Dimensional Imaging, Interaction, and Measurement*, ser. Proc. SPIE, 2011.

[53] A. Bhandari, A. Kadambi, R. Whyte, L. Streeter, C. Barsi, A. Dorrington, and R. Raskar, "Multifrequency time of flight in the context of transient renderings," in *ACM SIGGRAPH 2013 Posters*, 2013, p. 46.

[54] J. P. Godbaz, M. J. Cree, and A. A. Dorrington, "Closed-form inverses for the mixed pixel/multipath interference problem in AMCW lidar," in *Computational Imaging X*, ser. Proc. SPIE, 2012.

[55] D. Freedman, Y. Smolin, E. Krupka, I. Leichter, and M. Schmidt, "SRA: Fast removal of general multipath for ToF sensors," in *Proc. European Conference on Computer Vision (ECCV)*, 2014, pp. 234–249.

[56] N. Naik, A. Kadambi, C. Rhemann, S. Izadi, R. Raskar, and S. B. Kang, "A light transport model for mitigating multipath interference in time-of-flight sensors," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 73–81.

[57] A. Kadambi, R. Whyte, A. Bhandari, L. Streeter, C. Barsi, A. Dorrington, and R. Raskar, "Coded time of flight cameras: sparse deconvolution to address multipath interference and recover time profiles," *ACM Transactions on Graphics*, vol. 32, no. 6, p. 167, 2013.

[58] M. O'Toole, F. Heide, L. Xiao, M. B. Hullin, W. Heidrich, and K. N. Kutulakos, "Temporal frequency probing for 5D transient analysis of global light transport," *ACM Transactions on Graphics*, vol. 33, no. 4, p. 87, 2014.

[59] M. Gupta, S. K. Nayar, M. B. Hullin, and J. Martin, "Phasor imaging: A generalization of correlation-based time-of-flight imaging," *ACM Transactions on Graphics*, vol. 34, no. 5, p. 156, 2015.

[60] S. K. Nayar, G. Krishnan, M. D. Grossberg, and R. Raskar, "Fast separation of direct and global components of a scene using high frequency illumination," *ACM Transactions on Graphics*, vol. 25, no. 3, pp. 935–944, 2006.

[61] M. Gupta, A. Agrawal, A. Veeraraghavan, and S. G. Narasimhan, "Structured light 3D scanning in the presence of global illumination," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2011, pp. 713–720.

[62] M. Gupta and S. K. Nayar, "Micro phase shifting," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2012, pp. 813–820.

[63] T. Chen, H.-P. Seidel, and H. P. A. Lensch, "Modulated phase-shifting for 3D scanning," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2008, pp. 1–8.

[64] V. Couture, N. Martin, and S. Roy, "Unstructured light scanning to overcome interreflections," in *Proc. IEEE International Conference on Computer Vision (ICCV)*, 2011, pp. 1895–1902.

[65] E. Dedrick, "Improving SLI performance in optically challenging environments," Ph.D. dissertation, University of Kentucky, 2011.

[66] M. O'Toole, R. Raskar, and K. N. Kutulakos, "Primal-dual coding to probe light transport," *ACM Transactions on Graphics*, vol. 31, no. 4, pp. 39:1–39:11, 2012.

[67] M. O'Toole, J. Mather, and K. N. Kutulakos, "3D shape and indirect appearance by structured light transport," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2014, pp. 3246–3253.

[68] M. O'Toole, S. Achar, S. G. Narasimhan, and K. N. Kutulakos, "Homogeneous codes for energy-efficient illumination and imaging," *ACM Transactions on Graphics*, vol. 34, no. 4, pp. 35:1–35:13, 2015.
[69] Z. H. Zhang, C. E. Towers, and D. P. Towers, "Compensating lateral chromatic aberration of a colour fringe projection system for shape metrology," *Optics and Lasers in Engineering*, vol. 48, no. 2, pp. 159–165, 2010.
[70] J. Huang, Q. Xue, Z. Wang, and J. Gao, "Analysis and compensation for lateral chromatic aberration in a color coding structured light 3D measurement system," *Sensors*, vol. 16, no. 9, p. 1426, 2016.
[71] Y. Zhang, Y. Sun, N. Gao, Z. Meng, and Z. Zhang, "Phase target-based calibration of projector radial chromatic aberration for color fringe 3D measurement systems," *Sensors*, vol. 22, no. 18, p. 6845, 2022.
[72] J. Wang, Y. Yang, and Y. Zhou, "3-D shape reconstruction of non-uniform reflectance surface based on pixel intensity, pixel color and camera exposure time adaptive adjustment," *Scientific Reports*, vol. 11, p. 4700, 2021.
[73] Q. Zhu, H. Zhao, and Z. Gong, "A three-dimensional measurement method for colorful objects based on intensity pre-calibration," *Optical Review*, vol. 29, no. 4, pp. 343–353, 2022.
[74] D. Moreno and G. Taubin, "Simple, accurate, and robust projector-camera calibration," in *Proc. International Conference on 3D Imaging, Modeling, Processing, Visualization and Transmission (3DIMPVT)*, 2012, pp. 464–471.
[75] N. Matsuda, O. Cossairt, and M. Gupta, "MC3D: Motion contrast 3D scanning," in *Proc. IEEE International Conference on Computational Photography (ICCP)*, 2015.
[76] SparkFun Electronics, "Alchitry Pt V2," https://www.sparkfun.com/alchitry-pt-v2.html, accessed 2026-10-23.
[77] ——, "Alchitry Ft+ V2," https://www.sparkfun.com/alchitry-ft-plus-v2.html, accessed 2026-10-23.
[78] Future Technology Devices International, *FT600/FT601 USB 3.0 to FIFO Bridge Datasheet*, FTDI.
[79] SparkFun Electronics, "Alchitry Hd V2," https://www.sparkfun.com/alchitry-hd-v2.html, accessed 2026-10-23.
[80] Alchitry, "Alchitry Pt V2 schematic," power supply sheet.
[81] ——, "Alchitry Labs V2: board definitions and connector pin assignments," https://github.com/alchitry/Alchitry-Labs-V2, accessed 2026-10-23.
[82] Avnet, "MicroZed embedded vision kits: documentation and reference designs," https://community.element14.com/products/devtools/avnetboardscommunity/w/documents/3734/microzed-embedded-vision-kits, accessed 2026-10-23.
[83] Xilinx, "D-PHY solutions," Xilinx, Inc., Application Note XAPP894, 2021, v1.0.1.
[84] M. P. Ruffner, "Design of a machine vision camera for spatial augmented reality," Master's thesis, University of Kentucky, 2018. [Online]. Available: https://uknowledge.uky.edu/ece_etds/129/
[85] ——, "lupa300: FPGA camera for the LUPA300 image sensor," https://github.com/ruffner/lupa300.
[86] Analog Devices, *ADP5052: 5-Channel Integrated Power Solution with Quad Buck Regulators and 200 mA LDO Regulator*, Analog Devices.
[87] Texas Instruments, *TPS61023 3.7-A Boost Converter with 0.5-V Ultra-Low Input Voltage*, Texas Instruments.
[88] ——, *TPS7A20 300-mA, Ultra-Low-Noise, Low-IQ, High-PSRR Low-Dropout Linear Regulator*, Texas Instruments.
[89] ——, *TLV803S 3-Pin Voltage Supervisor with Active-Low, Open-Drain Reset*, Texas Instruments.
[90] M. Field, "Artix-7-HDMI-processing," https://github.com/hamsternz/Artix-7-HDMI-processing, MIT License.
[91] Embedded Micro, "Mojo HDMI shield schematic," https://www.tinyosshop.com/datasheet/Mojo-HDMI-Shield-Schematic.pdf, accessed 2026-10-23.
[92] Alchitry, "Alchitry Hd V2 schematic," https://cdn.alchitry.com/docs/Hd-V2/HdSchematic.pdf, accessed 2026-10-23.
[93] G. Meynants, "Global shutter pixels with correlated double sampling for CMOS image sensors," *Advanced Optical Technologies*, vol. 2, no. 2, pp. 177–187, 2013.
[94] T. Yokoyama, M. Tsutsui, M. Suzuki, Y. Nishi, I. Mizuno, and A. Lahav, "Development of low parasitic light sensitivity and low dark current 2.8 μm global shutter pixel," *Sensors*, vol. 18, no. 2, p. 349, 2018.
[95] A. Theuwissen, "Case study of a global shutter CIS—part 2: Parasitic light sensitivity," *IEEE Transactions on Electron Devices*, vol. 69, no. 6, pp. 2938–2942, 2022.